



西北工业大学
NORTHWESTERN POLYTECHNICAL UNIVERSITY

数字电子技术基础

实验报告

实验七：基于 FPGA 的模数转换电路

课程： 数字电子技术实验

组员 1: 郭浩然 2024303980

组员 2: 庞铭洋 2024302539

组号: 15

时间: 2026 年 6 月 15 日

1 实验目的

1. 理解高速模数转换芯片 AD9280 的工作原理，掌握它把 -5 V 至 $+5\text{ V}$ 模拟电压经衰减后转换成 0 至 255 的 8 位数字码的映射关系。
2. 掌握用计数器对板载时钟分频的方法，理解为什么要把 50 MHz 系统时钟分频后才能作为 AD9280 的驱动时钟。
3. 学会用 VHDL 编写显示译码模块，把 AD 转换得到的特征数据值译码到数码管上显示组员学号，并用 8 位 LED 直接显示数字码。
4. 熟悉 VHDL 文本设计与原理图设计相结合的混合开发流程，完整走通从建立工程到引脚分配、下载、外接 AD 模块和信号源测试的全过程。

2 实验要求

本次实验的要求以现场给出的实验内容为准，具体如下：

1. **要求 1：**用 FPGA 开发板和高速 A/D 模块实现模数转换。用信号源输出频率为 1 Hz 的方波信号，把方波信号接到 A/D 模块的模拟电压输入端，A/D 模块把模拟信号转换成数字信号。调节信号源幅度旋钮，使输出幅度分别在 $\pm 5\text{ V}$ 、 $\pm 4\text{ V}$ 、 $\pm 3\text{ V}$ 、 $\pm 2\text{ V}$ 、 $\pm 1\text{ V}$ 之间变化，依次记录正负峰值电压对应的 8 位数字输出 LED[7..0]。
2. **要求 2：**用信号源产生 1 Hz 方波，幅度调为 $+5\text{ V}$ 至 -5 V 变化。当信号为 $+5\text{ V}$ 输出时，两位数码管显示组里一名同学学号的后两位；当信号为 -5 V 输出时，两位数码管显示组里另一名同学学号的后两位；当信号源输出为其他幅度值时，两位数码管显示横线 ==。

本组规定： $+5\text{ V}$ 时数码管显示郭浩然学号后两位 80， -5 V 时显示庞铭洋学号后两位 39，其他所有输入幅度下两位数码管均显示横线 ==，表示当前没有有效的学号显示数据。

3 实验设备

- DE0-CV 开发板，FPGA 型号为 Cyclone V 5CEBA4F23C7
- 正点原子高速 AD/DA 子板，模数芯片为 AD9280，8 位 32 MSPS
- 函数信号发生器，提供 1 Hz 方波并可调幅度
- USB-Blaster 下载线及计算机

- Quartus II 开发软件
- 板载 50 MHz 晶振、8 位红色 LED、两位七段数码管及连接导线

4 实验原理

4.1 AD9280 模数转换原理

AD9280 是 ADI 公司的一款单芯片 8 位高速模数转换器，最高采样率 32 MSPS。它在时钟 CLK 的驱动下工作，内置采样保持放大器并采用多级差分流水线结构，从采样到输出数据要经过若干个时钟周期的流水延迟。芯片输出 8 位二进制码，并用 OTR 信号标志输入是否越出量程，量程内为低电平、越界为高电平。

AD9280 本身允许的模拟输入范围是 0 V 至 2 V，对应输出 0 至 255。子板在模拟输入端加了一级衰减电路，把外部 -5 V 至 +5 V 的信号 SMA_IN 衰减并平移成芯片可接受的 AD_IN2，二者关系为

$$AD_IN2 = \frac{SMA_IN}{5} + 1.$$

于是 SMA_IN = -5 V 时 AD_IN2 = 0 V、输出数据 0；SMA_IN = +5 V 时 AD_IN2 = 2 V、输出数据 255。把这两段比例合起来，外部电压 V 与输出数字码 D 之间近似为线性关系

$$D = 255 \times \frac{AD_IN2}{2} = 255 \times \frac{V + 5}{10} = 25.5(V + 5).$$

这就是后面用来核对实测数据的理论转换式。

4.2 时钟分频

AD9280 支持的最大时钟频率为 32 MHz，而 DE0-CV 板载系统时钟为 50 MHz，超过了芯片上限，必须先分频再作驱动时钟。本实验用一个 VHDL 计数器模块 freq_div 对 50 MHz 时钟分频，取二分频输出 clk_div_2 得到 25 MHz，接到 AD9280 的 ad_clk 作为转换驱动时钟，既满足小于 32 MHz 的限制，又有足够的采样速率跟踪 1 Hz 的方波。

4.3 数据显示与译码

AD 转换得到的 8 位数字码有两条去向。一条直接接到 8 个红色 LED D[7..0]，按二进制逐位点亮，用来直观读出转换结果的数值。另一条接到 VHDL 写的显示译码模块 trans，

由它根据特征数据值产生两位数码管的段码，把对应的字符显示出来。译码逻辑见表 1。

表 1: 显示译码模块 trans 的输入数据与数码管显示对照

输入数据 Data_IN	十进制	对应电压	数码管显示
11111111	255	+5 V 高电位	80, 郭浩然学号后两位
00000000	0	-5 V 低电位	39, 庞铭洋学号后两位
其他	其他	其他幅度	==, 无有效数据

4.4 系统结构

整个系统把文本设计和原理图设计结合起来：分频模块 `freq_div` 和译码模块 `trans` 用 VHDL 编写并各自生成符号，再在顶层原理图中调用并连线。信号流向是：信号源的 1 Hz 方波接 AD 子板的模拟输入，经衰减电路送 AD9280；FPGA 把 50 MHz 分频出的 25 MHz 通过 `outclk` 引脚送给 AD9280 作驱动时钟；AD9280 转换出的 8 位数据从 GPIO 回到 FPGA 的 `C[7..0]`，一路直接驱动 8 位 LED，另一路送 `trans` 译码后驱动两位数码管。顶层原理图见图 1。

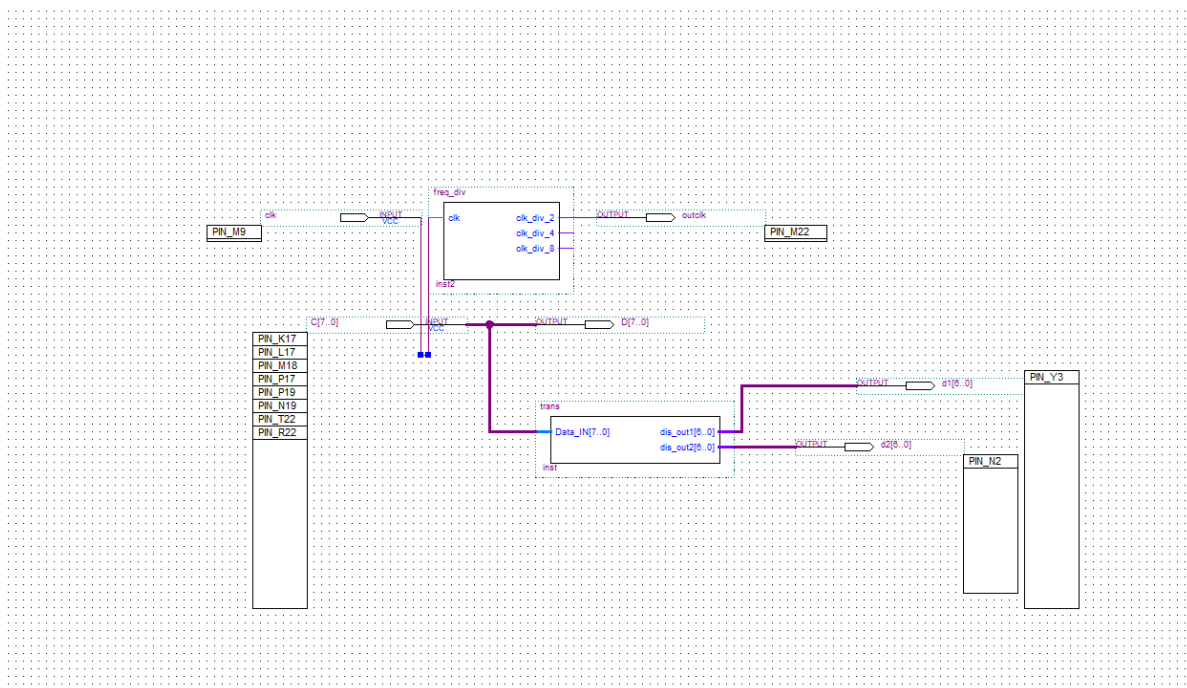


图 1: 基于 FPGA 的模数转换电路顶层原理图

5 实验内容

5.1 建立工程

打开 Quartus II, 用 **File** → **New Project Wizard** 新建工程, 器件选 DE0-CV 板对应的 Cyclone V 5CEBA4F23C7, 把顶层实体设为原理图文件 top。随后分别编写两个 VHDL 子模块并绘制顶层原理图。

5.2 编写分频模块 freq_div

freq_div 用一个 3 位计数器对输入时钟 clk 计数, 每个上升沿加 1、计满后归零, 再把计数器最低三位分别取反输出 clk_div_2、clk_div_4、clk_div_8, 对应二分频、四分频和八分频。本实验取二分频输出 clk_div_2, 把 50 MHz 变成 25 MHz 作 AD9280 的驱动时钟。

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.STD_LOGIC_ARITH.ALL;
4  USE IEEE.STD_LOGIC_UNSIGNED.ALL;
5  ENTITY freq_div IS
6  PORT (
7      clk          : IN  STD_LOGIC;
8      clk_div_2    : OUT STD_LOGIC;      -- 二分频, 25MHz
9      clk_div_4    : OUT STD_LOGIC;      -- 四分频
10     clk_div_8    : OUT STD_LOGIC);      -- 八分频
11 END freq_div;
12 ARCHITECTURE clk_div_behavior OF freq_div IS
13     SIGNAL counter : STD_LOGIC_VECTOR (2 DOWNTO 0);
14 BEGIN
15     PROCESS(clk)
16     BEGIN
17         IF(clk'EVENT AND clk = '1')THEN
18             IF(counter = "111")THEN counter <= "000";
```

```

19         ELSE counter <= counter + 1;
20     END IF;
21 END IF;
22 END PROCESS;
23 clk_div_2 <= NOT counter(0);
24 clk_div_4 <= NOT counter(1);
25 clk_div_8 <= NOT counter(2);
26 END clk_div_behavior;

```

Listing 1: 时钟分频模块 freq_div.vhd

5.3 编写译码模块 trans

trans 接收 AD 输出的 8 位数据 Data_IN，用一段 CASE 组合逻辑把特征值映射成两位数码管的段码 dis_out1、dis_out2，实现表 1 的显示。其中 255 显示 80、0 显示 39，其余所有值两位数码管均显示横线 ==。

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  ENTITY trans IS
4  port (Data_IN  : in  std_logic_vector (7 downto 0);
5        dis_out1 : out std_logic_vector (6 downto 0);
6        dis_out2 : out std_logic_vector (6 downto 0));
7  end trans;
8  ARCHITECTURE fwn OF trans IS BEGIN
9  PROCESS (Data_IN) BEGIN
10 CASE Data_IN IS
11     WHEN "11111111" => dis_out1 <= "1000000"; dis_out2 <= "0000000"; -- 高电
        ↳ 位 80
12     WHEN "00000000" => dis_out1 <= "0010000"; dis_out2 <= "0110000"; -- 低电
        ↳ 位 39
13     WHEN others      => dis_out1 <= "0111110"; dis_out2 <= "0111110"; -- 其他
        ↳ 情况显示 ==

```

```
14 END CASE;  
15 END PROCESS;  
16 END fwn;
```

Listing 2: 显示译码模块 trans.vhd

把这两个模块分别编译，再用 **File** → **Create/Update** → **Create Symbol Files for Current File** 各自生成 .bsf 符号，供顶层原理图调用。

5.4 绘制顶层原理图

新建一个 BDF 原理图作为顶层设计，把各模块符号放上去并按下面的关系连线，得到图 1。

- 输入有两个：clk 接板载 50 MHz 系统时钟，总线 C[7..0] 接 AD9280 输出的 8 位数据。
- freq_div 以 clk 为输入，二分频输出 clk_div_2 接到输出脚 outclk，作为 AD9280 的 25 MHz 驱动时钟。
- 输入总线 C[7..0] 一路直接接输出总线 D[7..0] 驱动 8 位 LED，另一路接 trans 的 Data_IN[7..0]。
- trans 的 dis_out1[6..0] 接输出 d1[6..0] 驱动第一位数码管，dis_out2[6..0] 接 d2[6..0] 驱动第二位数码管。

5.5 引脚分配

把 BDF 设为顶层实体完整编译一遍，再打开 Pin Planner 按表 2 把各端口锁定到 DE0-CV 板和 AD 子板对应的引脚上，引脚配置截图见图 2。其中 C[7..0] 接 GPIO 上 AD 子板的数据口 ad_data[7..0]，outclk 接 AD 子板的 ad_clk。

表 2: 引脚分配: 端口与 DE0-CV 板载资源及 AD 模块的对应关系

端口	方向	引脚	说明
clk	输入	PIN_M9	板载 50 MHz 系统时钟
C[7..0]	输入	K17,L17,M18,P17,P19,N19,T22,R22	AD9280 数据 ad_data[7..0]
outclk	输出	PIN_M22	25 MHz 时钟, 接 AD ad_clk
D[7..0]	输出	W2,Y3,N2,N1,U2,U1,L2,L1	8 位 LED, 显示 AD 数据
d1[6..0]	输出	U21,V21,W22,W21,Y22,Y21,AA22	第一位数码管 HEX0
d2[6..0]	输出	AA20,AB20,AA19,AA18,AB18,AA17,U22	第二位数码管 HEX1

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential
C[7]	Input	PIN_K17	5B	B5B_NO	PIN_K17	2.5 V (default)		12mA (default)		
C[6]	Input	PIN_L17	5B	B5B_NO	PIN_L17	2.5 V (default)		12mA (default)		
C[5]	Input	PIN_M18	5B	B5B_NO	PIN_M18	2.5 V (default)		12mA (default)		
C[4]	Input	PIN_P17	5A	B5A_NO	PIN_P17	2.5 V (default)		12mA (default)		
C[3]	Input	PIN_P19	5A	B5A_NO	PIN_P19	2.5 V (default)		12mA (default)		
C[2]	Input	PIN_N19	5B	B5B_NO	PIN_N19	2.5 V (default)		12mA (default)		
C[1]	Input	PIN_T22	5A	B5A_NO	PIN_T22	2.5 V (default)		12mA (default)		
C[0]	Input	PIN_R22	5A	B5A_NO	PIN_R22	2.5 V (default)		12mA (default)		
clk	Input	PIN_M9	3B	B3B_NO	PIN_M9	2.5 V (default)		12mA (default)		
D0	Output	PIN_W2	2A	B2A_NO	PIN_W2	2.5 V (default)		12mA (default)	1 (default)	
D1	Output	PIN_Y3	2A	B2A_NO	PIN_Y3	2.5 V (default)		12mA (default)	1 (default)	
D2	Output	PIN_N2	2A	B2A_NO	PIN_N2	2.5 V (default)		12mA (default)	1 (default)	
D3	Output	PIN_N1	2A	B2A_NO	PIN_N1	2.5 V (default)		12mA (default)	1 (default)	
D4	Output	PIN_U2	2A	B2A_NO	PIN_U2	2.5 V (default)		12mA (default)	1 (default)	
D5	Output	PIN_U1	2A	B2A_NO	PIN_U1	2.5 V (default)		12mA (default)	1 (default)	
D6	Output	PIN_L2	2A	B2A_NO	PIN_L2	2.5 V (default)		12mA (default)	1 (default)	
D7	Output	PIN_L1	2A	B2A_NO	PIN_L1	2.5 V (default)		12mA (default)	1 (default)	
d10	Output	PIN_U21	4A	B4A_NO	PIN_U21	2.5 V (default)		12mA (default)	1 (default)	
d11	Output	PIN_V21	4A	B4A_NO	PIN_V21	2.5 V (default)		12mA (default)	1 (default)	
d12	Output	PIN_W22	4A	B4A_NO	PIN_W22	2.5 V (default)		12mA (default)	1 (default)	
d13	Output	PIN_W21	4A	B4A_NO	PIN_W21	2.5 V (default)		12mA (default)	1 (default)	
d14	Output	PIN_Y22	4A	B4A_NO	PIN_Y22	2.5 V (default)		12mA (default)	1 (default)	
d15	Output	PIN_Y21	4A	B4A_NO	PIN_Y21	2.5 V (default)		12mA (default)	1 (default)	
d16	Output	PIN_AA22	4A	B4A_NO	PIN_AA22	2.5 V (default)		12mA (default)	1 (default)	
d20	Output	PIN_AA20	4A	B4A_NO	PIN_AA20	2.5 V (default)		12mA (default)	1 (default)	
d21	Output	PIN_AB20	4A	B4A_NO	PIN_AB20	2.5 V (default)		12mA (default)	1 (default)	
d22	Output	PIN_AA19	4A	B4A_NO	PIN_AA19	2.5 V (default)		12mA (default)	1 (default)	
d23	Output	PIN_AA18	4A	B4A_NO	PIN_AA18	2.5 V (default)		12mA (default)	1 (default)	
d24	Output	PIN_AB18	4A	B4A_NO	PIN_AB18	2.5 V (default)		12mA (default)	1 (default)	
d25	Output	PIN_AA17	4A	B4A_NO	PIN_AA17	2.5 V (default)		12mA (default)	1 (default)	
d26	Output	PIN_U22	4A	B4A_NO	PIN_U22	2.5 V (default)		12mA (default)	1 (default)	
outclk	Output	PIN_M22	5B	B5B_NO	PIN_M22	2.5 V (default)		12mA (default)	1 (default)	

图 2: Pin Planner 引脚分配截图

5.6 编译与下载

重新完整编译生成 .sof 配置文件, 接上 USB-Blaster, 用 JTAG 模式把配置文件下载到 DE0-CV 开发板。

5.7 接线与测试

把函数信号源的输出接到 AD 子板的模拟输入端 SMA_IN, AD 子板的 8 位数据口和 ad_clk 通过排针接到 FPGA 对应的 GPIO。信号源设为 1 Hz 方波, 先把幅度调到 $\pm 5\text{ V}$, 再依次减到 $\pm 4\text{ V}$ 、 $\pm 3\text{ V}$ 、 $\pm 2\text{ V}$ 、 $\pm 1\text{ V}$, 每一档分别在方波的正峰和负峰读出 8 位 LED 显示的二进制数字码, 同时观察两位数码管的显示。

5.8 实验数据

各档幅度下正负峰电压对应的 8 位数字输出实测值见表 3, 原始记录见图 3。方波幅度为 $\pm 5\text{ V}$ 时, 正峰和负峰对应的数码管显示和 LED 状态见图 4 的 (a) 和 (b); 其他输入幅度下数码管显示横线 ==, 见图 4 的 (c)。

表 3: 各档幅度下正负峰电压的 8 位数字输出实测值

信号源幅度	峰值电压	LED[7..0] 二进制	十进制
$\pm 5\text{ V}$	+5 V	11111111	255
	-5 V	00000000	0
$\pm 4\text{ V}$	+4 V	11101111	239
	-4 V	00010111	23
$\pm 3\text{ V}$	+3 V	11001101	205
	-3 V	00110001	49
$\pm 2\text{ V}$	+2 V	10110011	179
	-2 V	01001011	75
$\pm 1\text{ V}$	+1 V	10011001	153
	-1 V	01100101	101

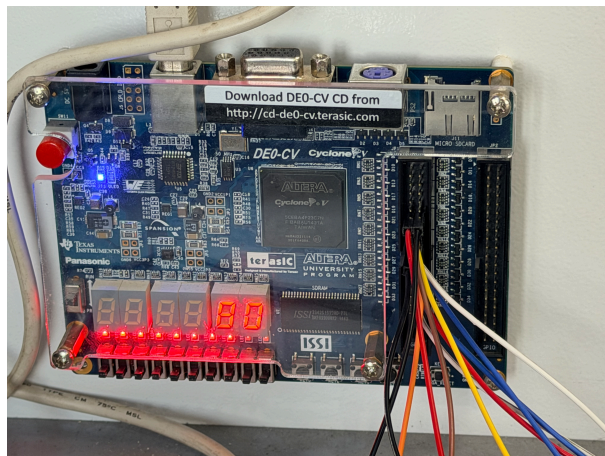
$\pm 5 \begin{cases} 255 \\ 0 \end{cases}$ $\pm 4 \begin{cases} 239 \\ 23 \end{cases}$

$\pm 3 \begin{cases} 11001101 \rightarrow 255 - 2 - 16 - 32 = 255 - 50 = 205 \\ 00110001 \rightarrow 49 \end{cases}$ 254

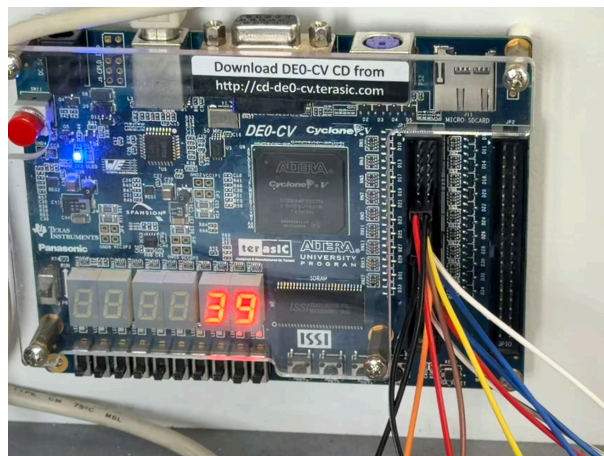
$\pm 2 \begin{cases} 10110011 \rightarrow 3 + 16 + 32 + 128 = 131 + 48 = 179 \\ 01001011 \rightarrow 1 + 2 + 8 + 64 = 64 + 11 = 75 \end{cases}$ 254

$\pm 1 \begin{cases} 10011001 \rightarrow 128 + 1 + 8 + 16 = 128 + 25 = 153 \\ 01100101 \rightarrow 1 + 4 + 32 + 64 = 65 + 36 = 101 \end{cases}$ 254

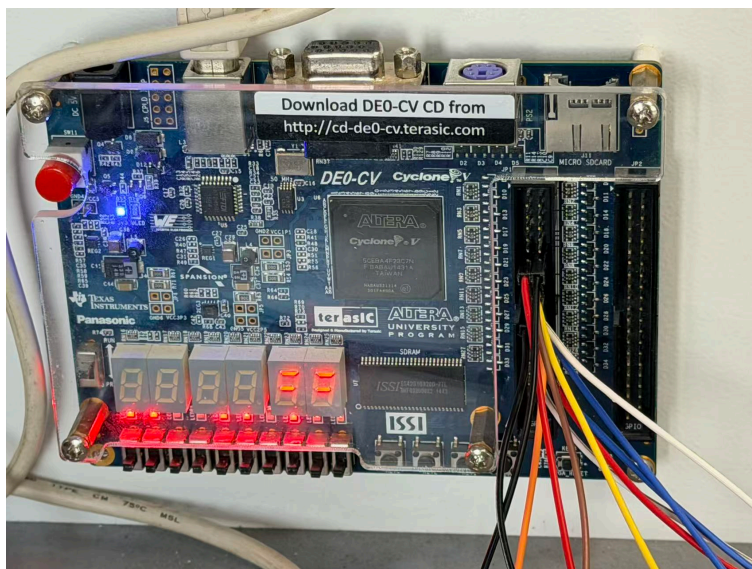
图 3: 8 位数字输出原始记录



(a) +5 V, 数码管显示 80, LED 全亮



(b) -5 V, 数码管显示 39, LED 全灭



(c) 其他幅度值, 数码管显示 ==

图 4: 两位数码管显示的实测结果

5.9 数据分析

按理论转换式 $D = 25.5(V + 5)$ 算出各电压对应的理论数字码, 与实测值对照并计算误差, 见表 4。

表 4: 实测数字码与理论值对照

峰值电压	理论值	实测值	误差
+5 V	255	255	0
-5 V	0	0	0
+4 V	230	239	+9
-4 V	26	23	-3
+3 V	204	205	+1
-3 V	51	49	-2
+2 V	179	179	0
-2 V	77	75	-2
+1 V	153	153	0
-1 V	102	101	-1

对照可以看出，实测数字码与理论值整体吻合很好，除 +4 V 档误差约 9 个最低有效位外，其余各档误差都在 3 个最低有效位以内。利用衰减电路的线性关系还可以做一个对称性检验：同一档正负峰的数字码理论上应满足 $D(+V) + D(-V) = 255$ 。实测中 $\pm 3\text{ V}$ 、 $\pm 2\text{ V}$ 、 $\pm 1\text{ V}$ 三档的正负峰之和都为 254，与 255 仅差一个量化位，进一步印证了转换的线性度。误差主要来自几个方面：信号源幅度旋钮的读数和实际输出本身有偏差，尤其手动调幅度时不易精确停在整伏；衰减电路的电阻和基准电压存在容差，使比例和平移略偏离理想值；AD9280 是 8 位量化，满量程 10 V 对应 256 级，每级约 39 mV，本身就有约一个最低有效位的量化误差；再加上方波峰值的人工读数误差。这些因素叠加在一起，使个别档位出现几个最低有效位的偏差，属正常范围。

数码管显示方面，+5 V 时正确显示 80、-5 V 时正确显示 39，分别对应两名组员学号的后两位；其他所有输入幅度下均显示横线 ==，与译码模块 `trans` 的设计要求一致。综合数字码和数码管两方面的结果，模数转换、时钟分频和显示译码三部分都工作正常，验证了系统设计的正确性。

6 实验过程中的问题

1. **驱动时钟必须分频。**AD9280 的最大转换时钟只有 32 MHz，直接用 50 MHz 板载时钟会超过上限导致转换不稳定。解决办法是用 `freq_div` 把 50 MHz 二分频成 25 MHz，再通过 `outclk` 引脚送给 AD9280 作驱动时钟。
2. **AD 数据线位序的对应。**AD9280 的数据口 `ad_data[7..0]` 要和 FPGA 输入总线 `C[7..0]` 一一对应，如果高低位接反，读出的数字码会变成镜像，电压和数据的对应关系全乱。接线时按表 2 的引脚逐根核对，确认低位对低位、高位对高位。
3. **衰减电路的电压标定。**外部 -5 V 至 $+5\text{ V}$ 必须经子板衰减电路变成 0 V 至 2 V 才能进 AD9280，关系是 $\text{AD_IN2} = \text{SMA_IN}/5 + 1$ 。一开始没注意这级衰减，按 0 V 至 5 V 估算数据对不上，弄清衰减关系后理论值才和实测吻合。
4. **译码特征值与学号的对应。**`trans` 里要把 255 译码成 80、把 0 译码成 39，调试时容易把两个数码管的段码或两名同学的学号对调，需要对照表 1 逐一核对段码，并在 $+5\text{ V}$ 和 -5 V 两种状态下实测确认显示正确。

7 心得体会

1. 这次实验让我把模数转换的完整链路理解透彻了。从信号源的模拟方波，到衰减电路把 $\pm 5\text{ V}$ 映射进 AD9280 的量程，再到芯片量化输出 8 位数字码，最后用 LED 和数码管显示，我清楚地看到了一个连续的模拟量是怎样一步步变成离散数字量的，也体会到衰减电路和量化精度对结果的直接影响。
2. 时钟分频这个环节看似简单，却很关键。正是因为 AD9280 有 32 MHz 的时钟上限，才必须先把 50 MHz 分频，这让我意识到接口芯片的时序约束在系统设计里不能忽视，时钟规划要从一开始就考虑进去。
3. 整个系统是 VHDL 文本设计与原理图设计的结合：分频和译码的逻辑用 VHDL 描述更清晰，各模块之间怎么连用原理图看得更直观。把它们生成符号再在顶层拼起来，配合外接 AD 模块和信号源测试，我完整走通了从数字设计到模数转换、再到结果显示的全过程，对数字系统的整体开发流程熟悉了不少。